

 Example 3: Edge case: Single block message (Advanced)

Learning Objective: Single block messages are simpler, Key truncation when key > block size, IV is still essential even for single blocks

Expected Final MAC: 0101

 Step 1: Single Block: XOR plaintext with IV

Input: 1010
Operation: XOR with IV (0011)
Calculation: 1010 $\oplus$ 0011 = 1001
Expected Result: 1001

[Next Step →](#) [← Previous Step](#) Step 1 of 2

 CBC-MAC Learning Examples

Choose from carefully crafted examples to understand CBC-MAC step by step:

[Example 1: Basic](#) [Example 2: Intermediate](#) [Example 3: Advanced](#) [Example 4: Complex](#)

[← Previous Example](#) [Next Example →](#) Or use random values: [Random Values](#)

Part 1: Basic CBC-MAC

Plaintext:

1010

[Next Plaintext](#)

Key (k):

11001100

[Next Key](#)

IV Length (l):

4

(where l < plaintext length/2)

IV:

0011

[Next IV](#)

 Test Function $F_k(\text{text})$:

This section allows you to test how the cryptographic function F_k transforms binary input. Enter binary text of the required length and see the step-by-step calculation.

 **Current Function F_k :** 1100

This determines how XOR (0) or EQV (1) operations are applied to each bit position.

Your text:

(Binary string of size 4)

1011

[Apply Function](#)

[Generate New Function](#)

 Function applied successfully!

Function output:
(F_k result)

Result will appear here after applying function

Function F_k Calculation

Details:

Input: 1011

Key (resized): 0011

Function pattern: 0111

Calculation:

- Position 0: 1 XOR 0
= 1
 - Position 1: 0 EQV 0
= 1
 - Position 2: 1 EQV 1
= 1
 - Position 3: 1 EQV 1
= 1
- Result: 1111

About Function Output:

This field shows the result when you apply the cryptographic function F_k to your input text. The function uses the current function pattern and key to transform your binary input into the corresponding output.

Final Output:

0101

Check Answer! Check (Original)

EXCELLENT! Your answer "0101" is correct!
You successfully calculated the CBC-MAC for Example 3

Part 2: Secure CBC-MAC

Select the option you would use to make CBC-MAC secure:

Prepend the message with its length

Plaintext:

10111001001000110111010011001001

Next Plaintext

Key:

11100100

Next Key

IV Length (I):

4 (where I < plaintext length/2)

IV:

1001

Next IV

Your Plaintext:

0010000010111001001000110111010011001001



Test Function F_k(text) - Secure CBC-MAC:

This section allows you to test how the cryptographic function F_k transforms binary input in the secure CBC-MAC implementation. Enter binary text of the required length and see the step-by-step calculation.

 Current Function F_k for Part 2: 1100

This determines how XOR (0) or EQV (1) operations are applied to each bit position in Secure CBC-MAC.

Your text:
(Binary string of size 4)

1100

Apply Function

Generate New Function

✓ Function applied successfully!

Function output:
(F_k result)

1110

 Secure CBC-MAC Function F_k Calculation Details:

Input: 1100
Key (resized): 1110
Function pattern: 1100
Calculation:

- Position 0: 1 EQV 1 = 1
- Position 1: 1 EQV 1 = 1
- Position 2: 0 XOR 1 = 1
- Position 3: 0 XOR 0 = 0

Result: 1110

 **About Secure CBC-MAC Function Output:** This field shows the result when you apply the cryptographic function F_k to your input text in the secure CBC-MAC implementation. The function uses the current function pattern and key to transform your binary input into the corresponding output.

Final Output:

0001

Check Secure CBC-MAC!

✓

Good choice! You selected: keys method for secure CBC-MAC. This helps prevent length extension and other attacks.